Adaptive Navigation and Conflict-Aware Path Planning

Robotics Software Engineer Hiring Assignment — Explanation of Approach

ROS 2 Humble · Gazebo Classic 11 · nav2 · colcon/ament

1. Design position

The brief asks for a cooperative navigation stack for a **heterogeneous** fleet. The word that shaped every decision here is *heterogeneous*: two robots with different mass, different sensing and different jobs, sharing one map and one piece of floor.

That leads to one governing rule, applied everywhere:

> **Every physical constant is declared once, in `robot_models.yaml`, and every consumer derives from it.**

A robot's maximum acceleration would ordinarily appear in five places — the nav2 controller limit, the velocity smoother, the Gazebo plugin ceiling, and hard-coded defaults in two custom nodes. Five copies drift the first time anyone tunes one. Here the model library is loaded by `amr_core::ModelLibrary` (C++) and `fleet_loader.py` (launch), and every downstream value is computed from it at launch time. Adding a third robot model is a YAML edit.

The second rule follows from the first:

> **Every claim in this repository is either derived or tested. Numbers are not restated in prose.**

Where a document quotes a dimension, a test reads that dimension from the generated world and fails the build if the prose is stale.

2. Workspace layout

Nine packages, split by responsibility rather than by convenience.

Package	Language	Responsibility
`amr_core`	C++	Model library, fleet config, geometry. No ROS dependency, so it is unit-testable in isolation.
`amr_msgs`	IDL	`PredictedTrajectory`, `SafetyStatus`, `YieldDirective`, `SetSafetyOverride`.
`amr_description`	xacro	One parameterised URDF; both robots are the same macro with different model data.
`amr_gazebo`	Python	**Generates** the world, the elevation map, the waypoints and the landmark manifest.
`amr_sensor_bsp`	C++	BSP validation for LiDAR, IMU and camera.
`amr_mapping`	C++	Map fusion and selective mapping.
`amr_navigation`	C++	`SlopeLayer` and `FleetTrajectoryLayer` nav2 costmap plugins.
`amr_fleet_control`	C++	Motion smoother, traffic control, safety override, trajectory broadcaster.
`amr_bringup`	Python	Launch, configuration, demos, consistency checks.

The world is generated, not drawn. `world_builder.py` emits the SDF, a PGM/YAML elevation map, `waypoints.yaml`, `dynamic_obstacles.yaml` and `warehouse_landmarks.txt` from one Python model. A hand-drawn world and a hand-written elevation map are two representations of the same geometry that diverge silently; the slope layer would then price ramps that are not where it thinks they are. Generating both from one source makes that class of bug impossible.

Current world: 34 static bodies, 4 ramps (5.0°, 6.0°, 6.0°, 9.0°), 2 elevated decks, 5 dynamic obstacles, 9 named goals, 44 × 30 m.

3. Requirement-by-requirement

3.1 Heterogeneous fleet (§1)

	AMR-1 `heavy_mapper`	**AMR-2** `light_scout`
Payload	120 kg	30 kg
Max linear velocity	0.75 m/s	1.4 m/s
Max linear acceleration	0.35 m/s^2	1.10 m/s^2
Max angular acceleration	0.80 rad/s^2	2.40 rad/s^2
LiDAR	1080 beams, 25 m	720 beams, 16 m
Yield priority	100	50

The heavy unit is the less agile one in **every** state, not just at rest — `DynamicLimits::EffectiveAccelX(load_ratio, speed_ratio)` sweeps the whole load/speed envelope and the test asserts the ordering across all of it.

3.2 Cooperative SLAM and map fusion (§2.1)

Each robot runs its own `slam_toolbox` in a private frame (`amr1/map`). `map_fusion_node` anchors each private frame into the shared `map` frame with a static transform derived from the robot's dock pose, then fuses contributions into a single occupancy grid published on `/map`.

Fusion is a **log-odds accumulator**, not a per-cell maximum. Taking the maximum means one robot's transient false positive — a pedestrian, a scan artefact — permanently marks a cell that the other robot can see is free. Log-odds lets disagreement resolve: repeated observation wins over a single sighting, and the accumulator is clamped to ±3.5 so no cell becomes unrevisable.

3.3 Selective mapping (§2.2, §2.3)

`SelectiveMappingPolicy` classifies every cell into three bands:

Band	**Condition**	**Update period**
Frontier	within `frontier_radius` (1.0 m) of unknown space	**always** — never throttled
Explored	visited fewer than `saturation_visits` (8) times	1.0 s
Saturated	visited at least 8 times	5.0 s

The visit count comes from **where the robot has actually driven**, not from where it has looked — a corridor traversed twenty times is saturated; a shelf face seen from twenty angles is not. On a fresh map almost everything is frontier and nearly all updates flow; on a mature map, suppression exceeds 90%. That is the "resource-efficient selective iteration" the brief asks for, and it is measured rather than asserted.

3.4 Slope-aware global planning (§2.5, §2.6)

`amr_navigation::SlopeCostModel` samples the generated elevation map, computes the local gradient, and prices it:

```
cost = base_cost + (max_cost − base_cost) · f(θ)^γ
```

with `base_cost = 200`, `max_cost = 252`, `γ = 1.0`, and lethal beyond each model's `max_traversable_angle_degrees`.

The subtle part is the ceiling. nav2 marks 253 as `INSCRIBED_INFLATED_OBSTACLE`, and `nav2_smac_planner`'s `Node2D::isNodeValid` **refuses** any cell at or above it. So 252 is expensive and 253 is impassable — there is no gradual approach to "never". A ramp priced 253 is not avoided, it is unusable, which silently breaks the other half of the requirement: that a ramp *is* taken when it is the only way through. The ceiling is therefore 252, and a test asserts it.

`base_cost = 200` with a linear curve is deliberate. A steep ramp prices itself out anyway; the 5° service ramp is the one that looks like a shortcut, and a squared curve is precisely what made it cheap. With the base high, the routing question is no longer *how steep* a cell is but *whether it is a ramp at all*.

Verified against the world, not asserted. A Dijkstra over the generated elevation map using Smac's own cell pricing (`cell × (1 + cost_travel_multiplier × cost/252)`):

```
dock_a → east_staging, doorway open   : 36.10   ramp used: no
dock_a → east_staging, doorway sealed : 54.15   ramp used: yes
dock_a → mezzanine_storage            : 46.03   ramp used: yes (reachable)
```

The planner accepts **18 m** of extra effective distance rather than climb, and still crosses the bridge the moment it is the only crossing. Both halves are asserted, because either alone is trivially satisfiable — make ramps free and the first fails, make them lethal and the second does.

3.5 Motion smoothing (§3.1–3.3)

`amr_fleet_control::MotionSmoother` implements a discrete-time S-curve terminal approach law. Acceleration and jerk are both bounded, and the limits are a function of **dynamic state** — current speed and current payload — via `DynamicLimits`, with payload injected at runtime through `SetPayload`.

The jerk clamp is applied **last** so it can only tighten the acceleration, never loosen it. Applying it before the acceleration clamp reintroduces exactly the discontinuity the S-curve exists to remove.

Note the layering: nav2's controllers have no concept of jerk. That is precisely why the smoother is a separate node downstream of the controller rather than a controller parameter.

3.6 Conflict-aware trajectory generation (§3.4, §3.5)

Two mechanisms, deliberately distinct:

Prediction. Each robot's `trajectory_broadcaster` publishes a 4 s projection at 10 Hz on `/fleet/trajectories`. Every robot's local costmap runs `amr_navigation::FleetTrajectoryLayer`, which writes **time-decayed** cost for its peers' predicted positions — present position at cost 200, the far end of the horizon at 30. A peer's predicted position is not an obstacle; it is a probability that decays with lookahead, and the cost should say so.

Arbitration. `traffic_control_node` runs a space-time `ConflictDetector` against all pairs and applies a priority `YieldPolicy`: the lighter AMR-2 always yields to the mission-critical AMR-1. The directive **scales the target velocity** rather than commanding a stop, so the motion smoother shapes the deceleration and the yield is a jerk-limited slow-down, not a cut.

The Pinch — a 2.10 m firewall doorway — exists so this fires. It is sized so two robots abreast fall inside the detector's conflict threshold (`r_a + r_b + margin` = 1.27 m), forcing the protocol rather than letting them squeeze past.

3.7 Safety override (§3.6–3.8)

`safety_override_node` is the **sole publisher of** `cmd_vel`. Everything upstream — nav2, the smoother, the traffic controller — publishes a *proposed* velocity on `cmd_vel_smoothed`. That single-writer arrangement is what makes the override real rather than advisory: there is no path by which an upstream command can reach the base while the monitor objects.

The envelope is speed-dependent, $d_{safe} = k \cdot v^2 + d_{min}$ (AMR-1: k = 0.85, d_min = 0.45 m). The decision runs **inside the scan callback**, not on a timer, so the reaction latency is the sensor period rather than the sensor period plus a scheduler quantum. `SafetyStatus.reaction_latency_us` records the interval from the scan's own acquisition stamp to the moment the halt was published.

The node **fails closed**: a stale scan halts the robot. Contrast the velocity smoother, which fails *open* — a smoother that stops publishing must not be able to freeze a robot that has no obstacle in front of it.

`check_model_consistency.py` verifies $k \cdot v_{max}^2 + d_{min} \le$ `lidar.range_max` for every model, because an envelope the sensor cannot see to is decorative.

3.8 BSP sensor validation (§4.1)

Not a HAL — a validation layer. `bsp_validation_node` subscribes to the raw Gazebo topics (`scan_raw`, `imu_raw`, `camera_raw/image_raw`) and republishes validated data on `scan`, `imu`, `camera`. **No part of the navigation stack subscribes to a raw topic**, so unvalidated data cannot reach a planner even by accident. nav2's costmaps, `slam_toolbox` and the safety monitor all consume the validated stream.

Faults are graded rather than binary — `LidarValidator`, `ImuValidator` and `CameraValidator` each distinguish *reject* from *degrade-and-forward*. The explicit requirement is met by `ImuValidator`: angular velocity beyond the model's `imu.max_angular_velocity` logs a warning naming the robot, the measured rate, the limit and the model. It is deliberately a **soft** fault — a 40 rad/s yaw reading on a warehouse AMR is far more likely to be a bad sample than a real event, and halting on it would be the wrong response.

3.9 Scalability (§4.2)

Every node is namespaced and every multi-robot structure is a container, not a pair of variables. `fleet_ten_robots.yaml` ships as proof: ten robots, no code change. The launch system builds its description from the roster at runtime, so fleet size is genuinely a configuration parameter.

A subtlety that cost real debugging time and is worth recording: `nav2_common.launch.RewrittenYaml` matches parameters **by key name**, replacing every occurrence in the file. That is correct for a radius and wrong for a frame — `global_frame` is `map` in the global costmap and `<robot>/odom` in the rolling local costmap. Frames and map extents therefore go through a **textual** `<ns>` / `<map_*>` substitution pass in `navigation.launch.py`, where each occurrence keeps its own value, and a guard test fails if a frame key ever reappears in the key-based rewrite.

4. Verification strategy

256 automated tests across six suites, plus a consistency checker and three demo scripts that exit non-zero on failure.

The method used throughout: **turn the symptom into a number and compare it against a physical or configuration quantity.** Examples where this located a real defect that the error message pointed away from:

Symptom	Number that explained it
`Starting point in lethal space!`	SLAM mapped a 19 × 13 cell box = 0.95 × 0.65 m — the chassis footprint. The LiDAR was mounted *inside* the robot.
`Robot is out of bounds of the costmap!`	Robot reported at (–37.0, 4.4); spawn x = –18.5. The pose was being anchored twice.
Robot stalls on ramps	Plant ceiling 1.40 rad/s^2 × 0.115 m = 0.16 m/s^2 against 1.03 m/s^2 of gravity on a 6° slope. `max_wheel_acceleration` is angular; it was set from a linear value.
Robot wedges between racks	2.40 m aisle – 2 × 0.75 m inflation = 0.90 m of usable lane, for a 1.10 m robot.
Scan develops fan-shaped voids	`sin(pitch)·cos(bearing)` = 0 sideways — one forward ground-return range was being applied to all 1080 beams.

Every fix is paired with a structural test, and every test is verified by reverting the fix and confirming the test fails. This mattered more than expected: on two occasions a guard passed its own negative control, which meant it was decoration. One compared the mapper's scan smear only *relative* to the scout's, which was satisfiable by making the mapper crawl; another set ratio floors *below* the ratios the fleet already shipped with.

A related discovery worth stating plainly: five C++ tests in `amr_navigation` had been failing silently because they hard-coded ramp probe points and gradients from an earlier version of the generated world. `colcon test` would have failed on submission. The fix was structural rather than a correction — `world_builder` now emits `maps/warehouse_landmarks.txt`, the C++ tests read it, and a Python test fails if the committed manifest goes stale. The same drift had reached the README and the traceability table, and is now guarded the same way.

5. Refactoring deliverable (§5.2)

Two areas identified, both implemented rather than proposed.

1. Parameter files → a typed C++ representation. The stock layout scatters a robot's constants across five files. `amr_core::ModelLibrary` makes the model data a typed struct loaded once and validated at load time, with a locating error message (`"<model>: key 'x' has the wrong type"`) rather than a bare `YAML::BadConversion`. Launch derives every nav2, Gazebo and smoother value from it.

2. A monolithic launch file → composable per-robot descriptions. `fleet.launch.py` → `robot.launch.py` → {`slam`, `navigation`, `fleet_control`}, with the roster driving how many robot groups exist. Robots are staggered by `spawn_delay` because spawning several models into Gazebo simultaneously wedges the spawn service, and bringing up N nav2 stacks at once trips lifecycle bonds.

`docs/REFACTORING.md` documents both in full, plus what a fuller version would tackle next.

6. Repository map

Path	Contents
`README.md`	Build and launch, step by step
`docs/RUNBOOK.md`	Operating procedures and a troubleshooting table indexed by symptom
`docs/DESIGN_NOTES.md`	Every non-obvious decision, and every bug worth recording, with the arithmetic
`docs/REQUIREMENTS.md`	Brief → implementation → evidence, one row per requirement
`docs/ARCHITECTURE.md`	Node graph, topic graph, TF tree
`docs/REFACTORING.md`	The §5.2 deliverable

`DESIGN_NOTES.md` is the document I would point a reviewer at first. It records the failures as well as the design — including the several occasions where a confident diagnosis was wrong, and what the evidence eventually showed. The debugging is the engineering.